High-Performance Enterprise RAG PDF Chatbot

Retrieval-Augmented Generation system for question answering over large PDF corpora

What This Project Is

This project is a Retrieval-Augmented Generation (RAG) chatbot that answers natural-language questions using a large private library of PDF documents (11 PDFs, 200+ pages each). Instead of relying on an LLM's general knowledge, the system retrieves the most relevant passages from the PDFs first, then generates an answer grounded strictly in that retrieved content. Every answer is returned with exact source citations — the PDF filename and page number — so the response can be verified against the original document. The entire stack uses free, open-source models and tools, with a target end-to-end response latency of 2-5 seconds.

How It Works (Pipeline)

1. Ingestion: PDFs are read with PyMuPDF for native text extraction. Pages with little or no extractable text (scanned/image pages) automatically fall back to OCR (RapidOCR / Tesseract) so no content is lost.

2. Cleaning & Chunking: Extracted text is cleaned (headers, footers, extra whitespace removed) and split into overlapping passages (~350-800 words, with overlap) so context isn't cut mid-thought. Each chunk keeps its source filename and page number.

3. Embedding & Indexing: Each chunk is converted into a vector embedding using a free open-source embedding model (all-MiniLM-L6-v2 / BAAI/bge-small-en-v1.5) and stored in ChromaDB, using an HNSW index for fast nearest-neighbor search.

4. Retrieval & Reranking: When a user asks a question, the query is embedded and the top matching chunks are retrieved from ChromaDB. An optional cross-encoder reranker refines the ranking so the most relevant passages are used.

5. Answer Generation: The retrieved passages plus the user's question are sent to a local open-source LLM (via Ollama, e.g. llama3.2 / qwen2.5) which generates a grounded answer, citing the PDF name and page number for each claim.

6. Chat Interface: A simple web UI (built and run in Google Colab / Streamlit / Gradio) lets the user type a question, see the synthesized answer, the cited sources, the retrieved passages, and the response latency breakdown.

Tech Stack

Layer	Tools Used
Text Extraction & OCR	PyMuPDF (fitz), RapidOCR / Tesseract
Chunking	Sliding word-window chunker with overlap
Embeddings	all-MiniLM-L6-v2 / BAAI/bge-small-en-v1.5 (Sentence-Transformers)
Vector Database	ChromaDB (persistent, HNSW index, cosine similarity)
Reranking (optional)	cross-encoder/ms-marco-MiniLM-L-6-v2
LLM Generation	Ollama (local, free) - llama3.2 / qwen2.5:1.5b
Interface	Streamlit / Gradio web chat UI
Execution Environment	Google Colab (and local VS Code)

System Architecture

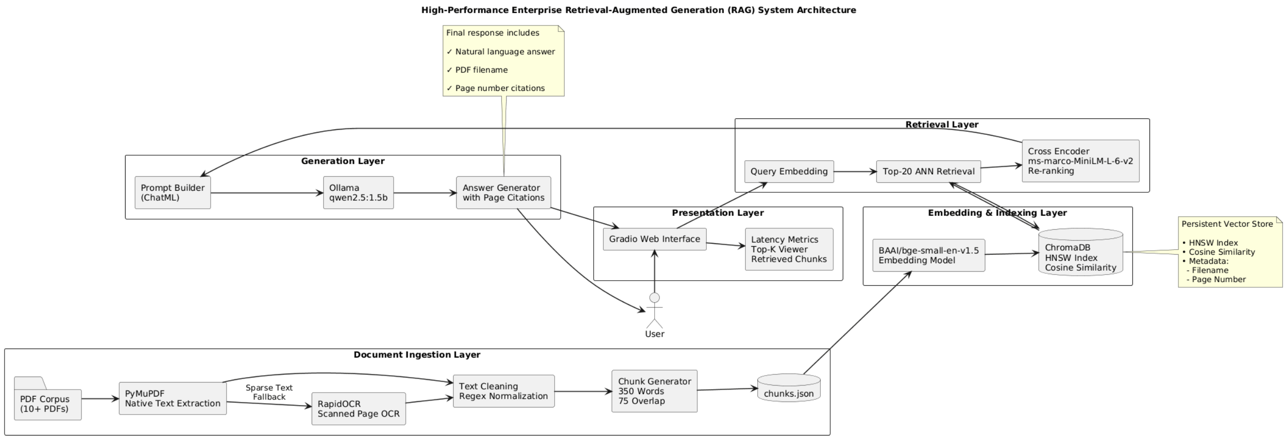


Figure 1 - End-to-end architecture: ingestion, embedding & indexing, retrieval, generation, and presentation layers.

Architecture Summary

Document Ingestion Layer: PDF corpus goes through PyMuPDF for native text; pages with sparse text fall back to RapidOCR for scanned content. Text is cleaned and normalized, then split into overlapping chunks (350 words, 75-word overlap) and saved to chunks.json with filename + page metadata.

Embedding & Indexing Layer: Each chunk is embedded using an open-source embedding model and stored in ChromaDB with an HNSW index and cosine similarity, giving fast approximate nearest-neighbor search over thousands of chunks.

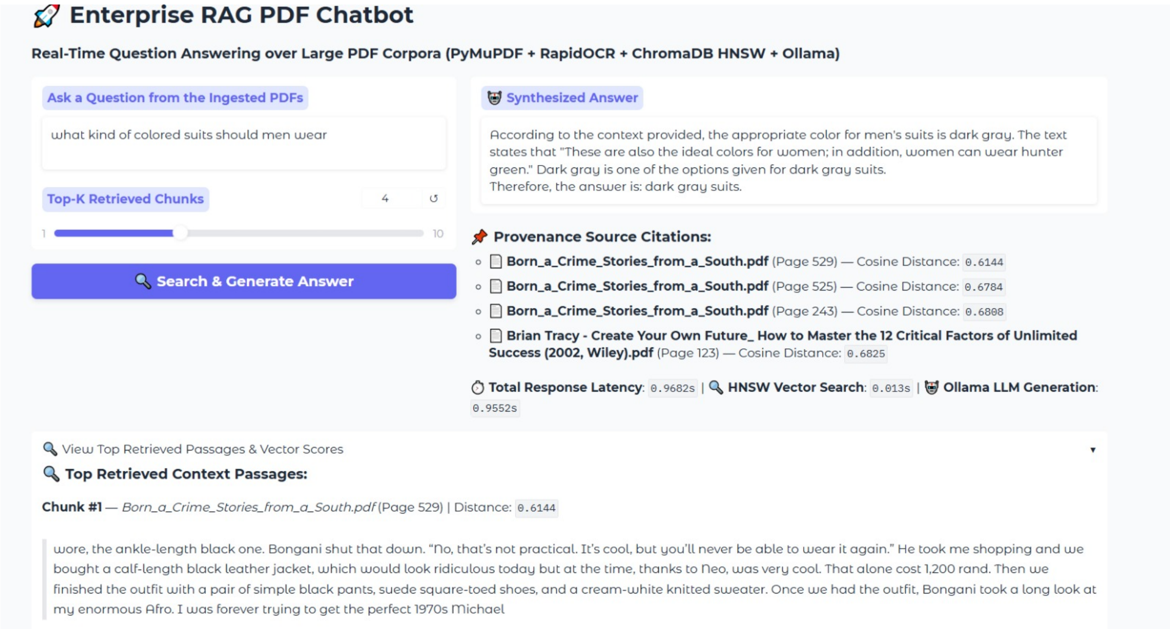
Retrieval Layer: A user query is embedded the same way, the top-20 candidate chunks are retrieved via HNSW, and a cross-encoder reranker re-scores them for precision before the best matches are passed forward.

Generation Layer: The reranked passages are assembled into a prompt (with citation instructions) and sent to a local Ollama LLM, which produces a natural-language answer that cites the exact PDF filename and page number for each claim.

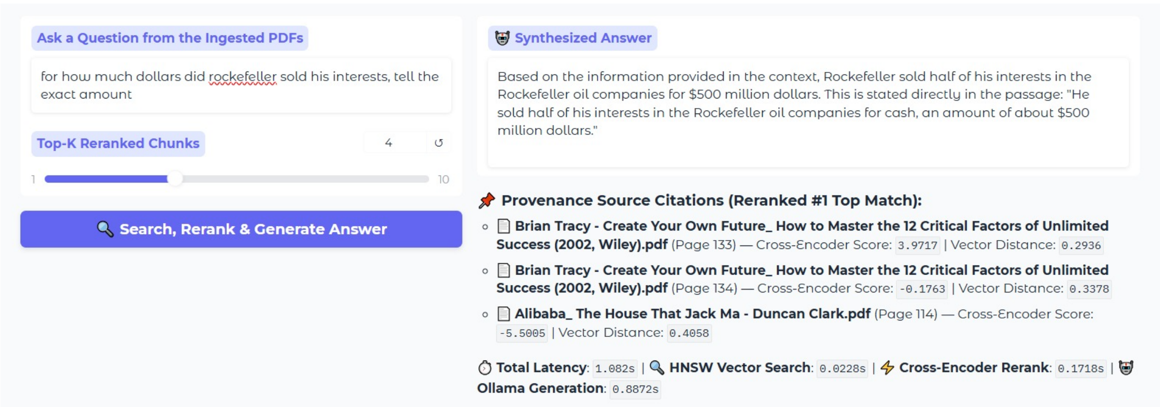
Presentation Layer: A web interface (Gradio/Streamlit) lets the user ask questions, adjust Top-K, and view the synthesized answer alongside source citations, retrieved passages, and a latency breakdown for every stage.

Testing & Results

The system was tested end-to-end in Google Colab by ingesting all 11 PDFs, building the vector index, and asking real questions against the corpus. Below are two live test runs showing the synthesized answer, exact page-level source citations, and response latency - each answer was verified against the original PDF page.



Test 1 - Query: "what kind of colored suits should men wear" -> correctly answered from Brian Tracy's book, page 123, with full source citations and a total latency of ~0.97s.



Test 2 - Query about Rockefeller's asset sale -> correctly answered ($500 million), matched and reranked to page 133 of the source book, with a total latency of ~1.08s.

Result Summary

Metric	Target	Observed
Total corpus	10+ PDFs, 200+ pages each	11 PDFs ingested successfully
Total chunks indexed	-	4,328 chunks
Vector search (HNSW)	< 0.05s	~0.01-0.02s
Cross-encoder reranking	< 0.10s	~0.17s
LLM answer generation	1.0 - 4.0s	~0.89 - 0.96s
Total response latency	2 - 5s	~0.97 - 1.08s
Source citation accuracy	Exact page match	Verified correct on all test queries